

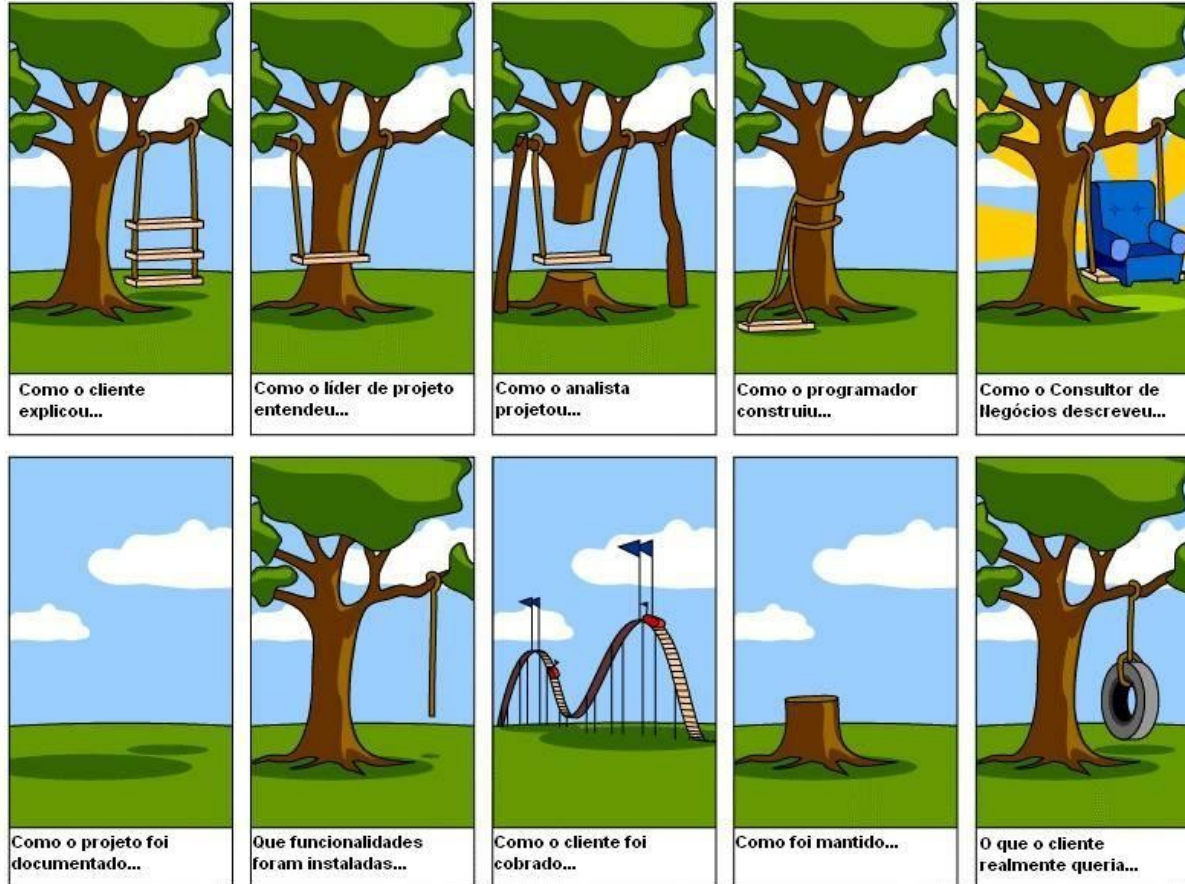
Projetos de Software 2

Uniube – Prof. Marcos Lopes



Projetos de Software 2

Uniube – Prof. Marcos Lopes



EMENTA

Técnicas de modelagem de sistemas orientados a objeto.

Estratégias de teste de software e ferramentas de garantia de qualidade de software.

Integração dos diagramas da linguagem UML.

Principais conceitos de arquitetura de software.

Desenvolvimento de um projeto de software orientado a objetos.

EMENTA:

- 1 - Técnicas de modelagem de sistemas orientados a objeto.
- 2- Estratégias de teste de software e ferramentas de garantia de qualidade de software.
- 3- Integração dos diagramas da linguagem UML.
- 4 - Principais conceitos de arquitetura de software.
- 5 - Desenvolvimento de um projeto de software orientado a objetos.

Objetivos específicos:

- Compreender os conceitos fundamentais de avaliação de software e qualidade.
- Aplicar as normas da ABNT voltadas à qualidade de software.
- Analisar o papel da engenharia da usabilidade na melhoria da qualidade de software.
- Utilizar mecanismos de avaliação para aplicação de ensaios de usabilidade.
- Identificar os componentes essenciais de um projeto de software.
- Elaborar projetos de software com foco na qualidade e na usabilidade.
- Reconhecer e aplicar modelos de gestão no acompanhamento de projetos de software.

Conteúdo programático:

1 - Técnicas de modelagem de sistemas orientados a objeto

1.1 - Projeto de software e seus componentes

1.2 - Elaborando um projeto de software

2 - Estratégias de teste de software e ferramentas de garantia de qualidade de software

2.1 - Qualidade de software

2.2 - Aplicando as normas da ABNT para qualidade de software

3 - Integração dos diagramas da linguagem UML

3.1 - Engenharia da usabilidade - colaborando com a qualidade de software

3.2 - Engenharia da usabilidade - mecanismos de avaliação para aplicação de ensaios de usabilidade

4 - Principais conceitos de arquitetura de software

4.1 - Identificando os modelos de gestão para aplicação e acompanhamento de um projeto de software

5 - Desenvolvimento de um projeto de software orientado a objetos

5.1 - Avaliação de software - uma visão geral

Qualidade de software:

coggle
made for free at coggle.it



Bugs Históricos:

Bugs Históricos:

- 1) MARINER I - 1962
 - 2) THERAC-25 - 1985/1987
 - 3) MISSIL PATRIOT - 1991
 - 4) PENTIUM DA INTEL - 1993
 - 5) BUG DO MILENIO (Y2K) - 2000
 - 6) TOYOTA PRIUS - 2010
 - 7) PLAY STATION NETWORK - 2011
 - 8) SISTEMA DE BAGAGENS (AEROPORTO DA ALEMANHA) - 2016
- O que é o bug? Por que aconteceu? Quais as consequências?

Bugs Históricos 1:

• **Mariner I – 1962**

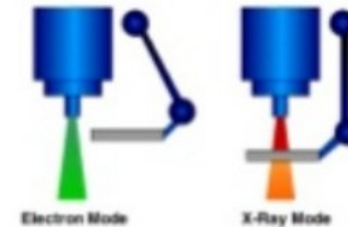
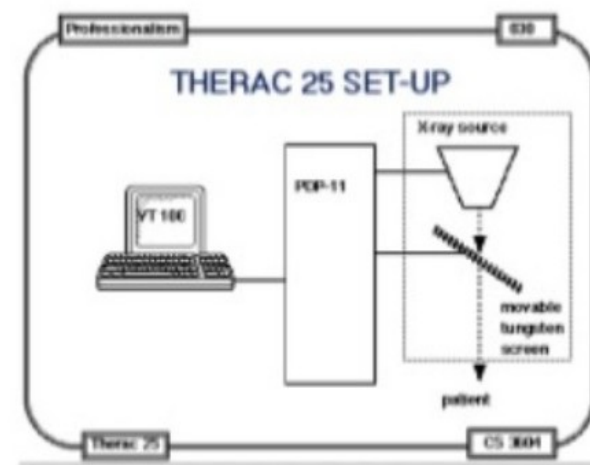
- Missão observar planeta Vênus
- Fórmula matemática foi equivocadamente transcrita para o computador
- Desviou do curso e foi destruído 4 min após lançamento
- Prejuízo: US\$ 18,5 mi



Bugs Históricos 2:

• Therac-25 – 1985/1987

- Dispositivo de terapia por radiação sobre células cancerosas
- Libera doses letais de radiação em vários consultórios médicos
- Condição de disputa no SO
- 5 mortes, várias pessoas feridas



Bugs Históricos 3:

- **Míssil Patriot – 1991**

- Míssil de defesa
- Erro de software no relógio do míssil: a cada 100 horas o relógio interno do sistema desviava um terço de segundo
- Resultado: 600 metros de erro na distância em uma interceptação
- 28 soldados americanos mortos



Bugs Históricos 4:

- **Pentium da Intel – 1993**

- Erro na divisão de pontos flutuantes (erro ~0,006%)
- 3 a 5 milhões de peças com defeito
- Recall para todos que quiseram trocar
- Custou à Intel US\$ 475 milhões



$$\frac{4195835}{3145727} = 1.333820449136241002$$

$$\frac{4195835}{3145727} = 1.333739068902037589$$

Bugs Históricos 5:

- **Bug do milênio (Y2K) – 2000**
 - Datas com apenas 2 dígitos para o ano
 - Uma das maiores histerias da história
 - Ao virar o ano 2000, a preocupação era que contasse como 1900
 - Entre US\$ 300 e US\$ 500 bi no mundo todo



Bugs Históricos 6:

- **Toyota Prius – 2010**

- Problema no software dos freios ABS
- Recall de 400.000 veículos
- Prejuízo de ~ US\$ 2 bilhões, desvalorização de 15% nas ações



Bugs Históricos 7:

- **Play Station Network - 2011**

- Invasão do sistema
- Dados privados e de cartão de crédito de ~70 mi de pessoas foram roubados



Bugs Históricos 8:

- **Sistema de bagagens (aeroporto Alemanha) - 2016**
 - Malas retidas (ano bissexto)
 - Sistema de 70 milhões de euros não reconheceu o 29 de fevereiro.
 - Esteiras pararam – despacho manual de bagagens transportadas entre 25 mil e 50 mil malas por dia!.
 - 1.200 malas acabaram não sendo despachadas

Avaliações:

M1: 35 (25 avaliação, 5 atividade, 5 uniube+) 07/04 até 17/04

M2: 50 (30 avaliação, 10 atividade, 10 uniube+) 17/06 até 24/06

4 Checkpoints avaliativos:

- 19/03 avaliação (checkpoint 1): valor 10 pontos
- 16/04 avaliação (checkpoint 2): valor 15 pontos
- 14/05 avaliação (checkpoint 3): valor 10 pontos
- 18/06 avaliação (checkpoint 4): valor 20 pontos

+ 15 de atividades +15 de uniube mais + 15 avaliação intitucional

- * O que são os checkpoints? validações da evolução dos trabalhos da disciplina
- * Grupos de no máximo 5 alunos
- * O checkpoint será feito por meio de apresentação de cada grupo no dia especificado acima.
- * Entrega do material (documento do projeto e artefatos) pelo diário de bordo até a data dos checkpoints.

Projeto da disciplina de “Projetos de software 2”:

1- Objetivo: construir um sistema/software que

- Tenha pelo menos duas interfaces com o usuário (web e mobile);
- Que seja acessível remotamente (implantado nas nuvens).
- Tem que ser um projeto o qual não exista código legado (desenvolver durante o semestre)!
- Pode já existir a documentação parcial.
- Tem que ter no mínimo 8 entidades no modelo de dados.

2- Entregas (artefatos)

- o projeto do software (documentação técnica)
- o software funcionando de acordo com o projeto
- manual do usuário (documentação do usuário)

3 – Datas

- definidas nos checkpoints

Projeto “final” da disciplina de “Projetos de software 2”:

4- Etapas

4.1- definição do escopo do projeto

4.2- levantamento de requisitos

- funcionais

- não funcionais

4.3- construção do(s) diagrama(s) de casos de uso

4.4 - quadro de gestão das atividades



Checkpoint 1

4.5- prototipação de telas (Figma)

4.6- construção dos documentos de casos de uso

4.7- construção do diagrama de dados (DER)

4.8- construção do diagrama de classes do modelo de dados



Checkpoint 2

Projeto “final” da disciplina de “Projetos de software 2”:

4.7- implementação dos serviços de backend (REST)

- definir tecnologia de backend
- usar TDD
- usar Git
- implantar nas nuvens

4.8- construção do diagrama de classes das principais interações entre componentes do sistema



Checkpoint 3

4.9- construção do sistema web (frontend)

- React / Angular / etc?
- implantar nas nuvens

Projeto “final” da disciplina de “Projetos de software 2”:

4.10- construção do sistema mobile (app)

- React-native / flutter / android nativo / etc?
- disponibilizar pra download nas nuvens

4.11- documentar o manual do usuário



Checkpoint 4

